Retail Sales Database

DS604 Introduction to Data Management - Autumn 2026

Milestone 2: ER Diagram, Relational Schema and DDL Scripts

Student ID	Group member
202618057	Pratham Patel
202618058	Rohan Deshpande
202618040	Hiteshsinh Rathod

1. Dataset Overview

The dataset models the operations of a multi-city retail chain: customers place orders at stores, each order contains items drawn from a product catalogue, and orders are paid, shipped and sometimes returned. It has **12 tables** and 1,591,380 rows in total (the project requires at least 5 tables). Source files are CSV, one per table.

Table	Rows	Description	Primary key
categories	30	Product categories	category_id
suppliers	200	Suppliers and their country (India, USA, China)	supplier_id
stores	100	Retail stores and city (Mumbai, Delhi, Pune, Bangalore)	store_id
promotions	50	Discount campaigns; discount is a percentage (5 to 39)	promotion_id
customers	50,000	Customer city and signup date	customer_id
employees	1,000	Employees, their store and salary	employee_id
products	10,000	Product catalogue: category, supplier, list price	product_id
orders	300,000	Order header: customer, store, date, promotion	order_id
order_items	600,000	Order lines: product, quantity, unit price at sale	order_item_id
payments	300,000	Payment recorded against an order	payment_id
shipments	300,000	Shipment status per order (shipped / delivered / late)	shipment_id
returns	30,000	Refund recorded against an order line	return_id

Schema at a glance

Item	Count
Tables	12
Columns	37 (36 are NOT NULL; only orders.promotion_id allows NULL)
Primary keys	12 (all single-column)
Foreign keys	11
UNIQUE constraints	1 (categories.category_name)
CHECK constraints	8
Secondary indexes	12 (on foreign-key columns and orders.order_date)
Trial load	All 12 CSV files (1,591,380 rows) loaded with every primary key, foreign key and CHECK constraint enforced; no row was rejected

2. Design Decisions and Assumptions

- **Keys.** Every table has a single-column integer primary key taken from the CSV. All 11 foreign keys were verified against the data: there are no orphan rows, so every constraint can be enforced.
- **Money columns** use NUMERIC (exact decimals) rather than floating point. The CSVs hold whole numbers, which load into NUMERIC without change.
- **orders.promotion_id is nullable.** Every order in the data has a promotion, but in a real business an order may have none, so NULL means "no promotion".
- **payments and shipments are 1:N from orders.** The data has exactly one of each per order, but real orders can have split payments or split shipments, so we did not add UNIQUE on order_id.
- **returns is 1:N from order_items.** 731 order lines appear in returns more than once, so order_item_id cannot be UNIQUE there.
- **order_items.price** is the unit price at the time of sale. It differs from products.price in almost every row (prices change), so it is stored separately and is not derived.
- **CHECK constraints** restrict discount to 0-100, qty above 0, money values non-negative, and shipment status to shipped / delivered / late.
- **Indexes** are created on every foreign key and on orders.order_date, because the analytical queries join and filter the 300k and 600k row tables heavily.

Data-quality observations (found while profiling)

The data is clean structurally (no NULLs, no duplicate rows, no orphan keys) but some business rules do not hold. We deliberately did not add constraints for these, since the load would fail. We analyse them in the query milestone instead.

Observation	Count
Orders with no order_items rows	40,767 of 300,000
Orders whose payment amount equals the sum of their items	10 of 259,233 (payments are unrelated to item totals)
Orders dated before the customer signup_date	120,020 (40.0% of orders)
Returns where refund exceeds qty x price of the line	7,544 of 30,000 (25.1%)
Order lines with more than one return row	731

3. Candidate Keys, Functional Dependencies and Normal Form

A **candidate key** is a minimal set of columns that uniquely identifies every row. One candidate key per table is chosen as the primary key (PK). A **functional dependency** X -> Y means that each value of X determines exactly one value of Y. The dependencies below come from what the columns mean in the business; we then checked them against the data (a data check can disprove a dependency but cannot prove one).

Table	Candidate key(s)	Functional dependencies	Normal form
categories	**{category_id}** (PK) {category_name} (UNIQUE)	category_id -> category_name category_name -> category_id Both determinants are candidate keys.	BCNF
suppliers	**{supplier_id}** (PK)	supplier_id -> country country is not unique (3 values for 200 suppliers), so it is not a key.	BCNF
stores	**{store_id}** (PK)	store_id -> city	BCNF
promotions	**{promotion_id}** (PK)	promotion_id -> discount discount is not unique (27 distinct values for 50 promotions), so it is not a key.	BCNF
customers	**{customer_id}** (PK)	customer_id -> city, signup_date	BCNF
employees	**{employee_id}** (PK)	employee_id -> store_id, salary	BCNF
products	**{product_id}** (PK)	product_id -> category_id, supplier_id, price	BCNF
orders	**{order_id}** (PK)	order_id -> customer_id, store_id, order_date, promotion_id	BCNF
order_items	**{order_item_id}** (PK)	order_item_id -> order_id, product_id, qty, price {order_id, product_id} is *not* a key: the same product can appear on two lines of one order.	BCNF
payments	**{payment_id}** (PK)	payment_id -> order_id, amount order_id is unique in the current data, but we do not declare it a key (see note 3).	BCNF
shipments	**{shipment_id}** (PK)	shipment_id -> order_id, status Same note as payments.	BCNF
returns	**{return_id}** (PK)	return_id -> order_item_id, refund order_item_id is not a key: 731 order lines have more than one return.	BCNF

Notes on the keys

1. Every candidate key is a single column, so each one is automatically minimal. Only categories has two candidate keys.
2. **Coincidental uniqueness is not a key.** In the current data the pairs (store_id, salary) in employees and (order_item_id, refund) in returns happen to be unique. Nothing in the business prevents two employees of one store from earning the same salary, so we do not treat them as keys.
3. **payments and shipments.** order_id is unique in both tables in the current data. We kept payment_id and shipment_id as the only keys because a real order can have split payments or split shipments. Making order_id UNIQUE would be the only change needed if the business rule were one payment and one shipment per order.
4. **order_items.price is not a copy of products.price.** It is the price at the time of sale. Every product in the data was sold at more than one unit price, so product_id does not determine price inside order_items. This is why the column is stored in both tables without breaking 3NF.

How each normal form was checked

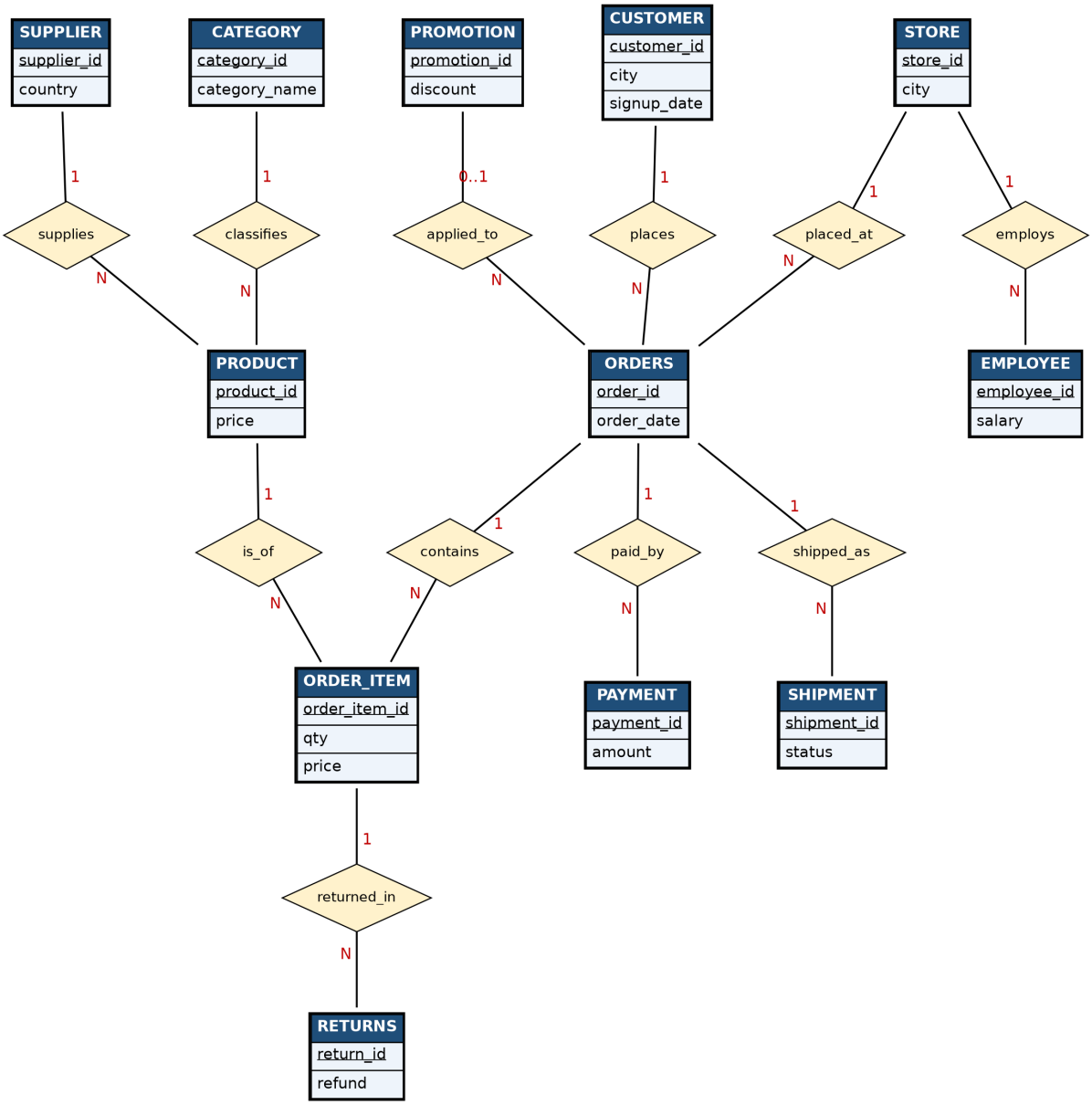
- **1NF.** Every column holds one atomic value, there are no repeating groups or lists, and every table has a primary key.
- **2NF.** All 12 primary keys are single columns, so no non-key column can depend on only part of a key. Every table in 1NF is therefore in 2NF.
- **3NF.** No non-key column determines another non-key column. Facts that belong to another entity live in that entity's table: the discount is in promotions (orders stores only promotion_id), the category name is in categories (products stores only category_id), the country is in suppliers, and the store city is in stores. The order total is deliberately not stored, because it can be calculated as SUM(qty x price) over the order lines and a stored copy could become inconsistent. payments.amount is kept as its own fact, since it is a recorded payment and does not equal the item total in the data.

- **BCNF.** For every non-trivial dependency in the table above, the determinant is a candidate key. We also tested every single-column determinant in all 12 CSV files: no non-key column determines any other column, which agrees with the dependencies we derived.

Result: all 12 tables are in BCNF, which also means they are in 3NF, 2NF and 1NF. The only redundancy in the design is the foreign keys themselves, which are required to link the tables.

4. ER Diagram

Entities show their own attributes only (primary key underlined). Foreign keys appear in the relational schema, not here, since in the ER model they come from the relationships.



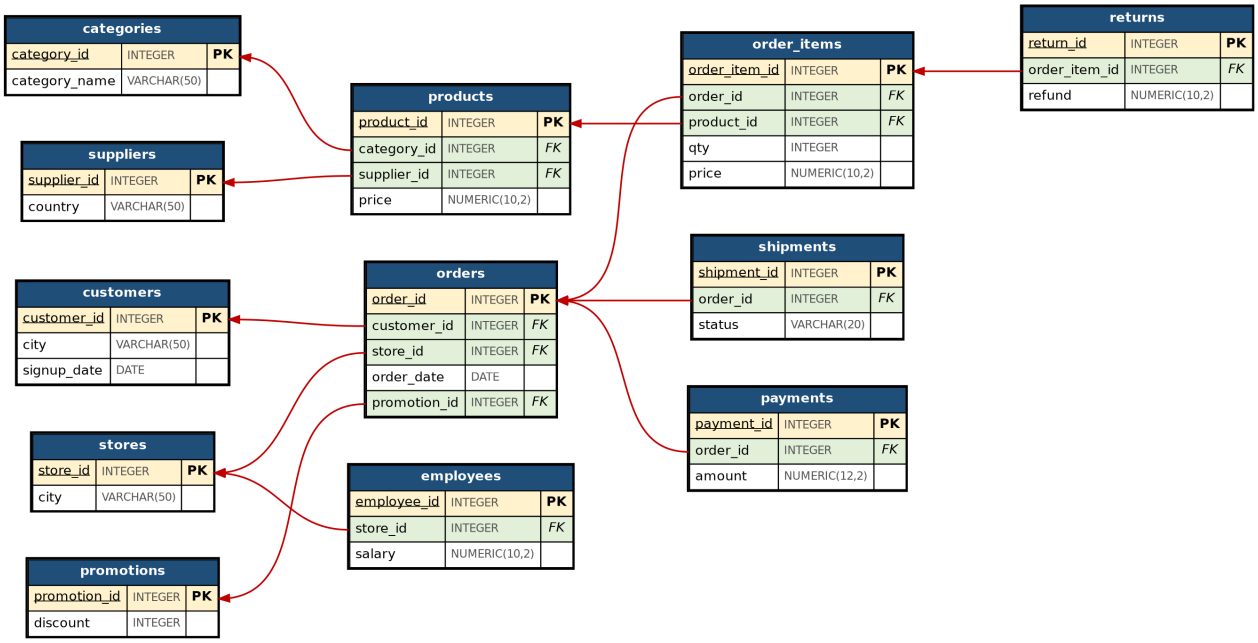
Relationships and cardinalities

Relationship	Entity A	Entity B	Cardinality	Participation of B
places	CUSTOMER	ORDERS	1 : N	Total (every order has a customer)
placed_at	STORE	ORDERS	1 : N	Total
employs	STORE	EMPLOYEE	1 : N	Total
applied_to	PROMOTION	ORDERS	1 : N	Optional on the order side (0..1 promotion)
contains	ORDERS	ORDER_ITEM	1 : N	Total
is_of	PRODUCT	ORDER_ITEM	1 : N	Total
paid_by	ORDERS	PAYMENT	1 : N	Total
shipped_as	ORDERS	SHIPMENT	1 : N	Total
returned_in	ORDER_ITEM	RETURNS	1 : N	Total
classifies	CATEGORY	PRODUCT	1 : N	Total
supplies	SUPPLIER	PRODUCT	1 : N	Total

All relationships are one-to-many. The many-to-many link between ORDERS and PRODUCT is resolved by the ORDER_ITEM entity.

5. Relational Schema Diagram

Yellow rows are primary keys, green rows are foreign keys. Each arrow points from a foreign key to the primary key it references.



Schema in relational notation

categories (category_id, category_name)

suppliers (supplier_id, country)

stores (store_id, city)

promotions (promotion_id, discount)

customers (customer_id, city, signup_date)

employees (employee_id, store_id^{FK}, salary)

products (product_id, category_id^{FK}, supplier_id^{FK}, price)

orders (order_id, customer_id^{FK}, store_id^{FK}, order_date, promotion_id^{FK})

order_items (order_item_id, order_id^{FK}, product_id^{FK}, qty, price)

payments (payment_id, order_id^{FK}, amount)

shipments (shipment_id, order_id^{FK}, status)

returns (return_id, order_item_id^{FK}, refund)

6. DDL Scripts (PostgreSQL)

Run in this order. Tables are created parents first, so foreign keys resolve. The DROP statements at the top make the script safe to re-run.

```
-- ========================================================================
-- DS604 Group Project : Retail Sales Database (PostgreSQL)
-- Milestone 2 : DDL Script
-- Tables are created in dependency order (parents before children).
-- ========================================================================
DROP TABLE IF EXISTS returns     CASCADE;
DROP TABLE IF EXISTS shipments   CASCADE;
DROP TABLE IF EXISTS payments    CASCADE;
DROP TABLE IF EXISTS order_items CASCADE;
DROP TABLE IF EXISTS orders      CASCADE;
DROP TABLE IF EXISTS products    CASCADE;
DROP TABLE IF EXISTS employees   CASCADE;
DROP TABLE IF EXISTS customers   CASCADE;
DROP TABLE IF EXISTS promotions CASCADE;
DROP TABLE IF EXISTS stores      CASCADE;
DROP TABLE IF EXISTS suppliers   CASCADE;
DROP TABLE IF EXISTS categories CASCADE;
-- ---------- Independent (parent) tables ----------
CREATE TABLE categories (
    category_id    INTEGER           PRIMARY KEY,
    category_name VARCHAR(50)        NOT NULL UNIQUE
);
CREATE TABLE suppliers (
    supplier_id    INTEGER           PRIMARY KEY,
    country        VARCHAR(50)       NOT NULL
);
CREATE TABLE stores (
    store_id       INTEGER           PRIMARY KEY,
    city           VARCHAR(50)       NOT NULL
);
CREATE TABLE promotions (
    promotion_id   INTEGER      PRIMARY KEY,
    discount       INTEGER      NOT NULL
                   CHECK (discount BETWEEN 0 AND 100)            -- percentage
);
CREATE TABLE customers (
    customer_id    INTEGER           PRIMARY KEY,
    city           VARCHAR(50)       NOT NULL,
    signup_date    DATE              NOT NULL
);
-- ---------- Tables with one level of dependency ----------
CREATE TABLE employees (
    employee_id    INTEGER        PRIMARY KEY,
    store_id       INTEGER        NOT NULL
                   REFERENCES stores(store_id),
    salary         NUMERIC(10,2) NOT NULL CHECK (salary > 0)
);
CREATE TABLE products (
    product_id     INTEGER        PRIMARY KEY,
    category_id    INTEGER        NOT NULL
                   REFERENCES categories(category_id),
    supplier_id    INTEGER        NOT NULL
                   REFERENCES suppliers(supplier_id),
    price          NUMERIC(10,2) NOT NULL CHECK (price >= 0)
);
CREATE TABLE orders (
    order_id       INTEGER      PRIMARY KEY,
    customer_id    INTEGER      NOT NULL
                   REFERENCES customers(customer_id),
    store_id       INTEGER      NOT NULL
                   REFERENCES stores(store_id),
    order_date     DATE         NOT NULL,
    promotion_id   INTEGER
                   REFERENCES promotions(promotion_id)           -- NULL = no promotion applied
);
-- ---------- Tables that depend on orders / order_items ----------
CREATE TABLE order_items (
    order_item_id INTEGER         PRIMARY KEY,
    order_id       INTEGER        NOT NULL
                   REFERENCES orders(order_id),
    product_id     INTEGER        NOT NULL
                   REFERENCES products(product_id),
    qty            INTEGER        NOT NULL CHECK (qty > 0),
    price          NUMERIC(10,2) NOT NULL CHECK (price >= 0)         -- unit price at time of sale
);
CREATE TABLE payments (
    payment_id     INTEGER        PRIMARY KEY,
    order_id       INTEGER        NOT NULL
                   REFERENCES orders(order_id),
    amount         NUMERIC(12,2) NOT NULL CHECK (amount >= 0)
);
CREATE TABLE shipments (
    shipment_id    INTEGER      PRIMARY KEY,
    order_id       INTEGER      NOT NULL
                   REFERENCES orders(order_id),
```

```
    status         VARCHAR(20) NOT NULL
                   CHECK (status IN ('shipped', 'delivered', 'late'))
);
CREATE TABLE returns (
    return_id      INTEGER        PRIMARY KEY,
    order_item_id INTEGER         NOT NULL
                   REFERENCES order_items(order_item_id),
    refund         NUMERIC(10,2) NOT NULL CHECK (refund >= 0)
);
-- ---------- Indexes on foreign keys (speeds up joins on 300k-600k row tables) ----------
CREATE INDEX idx_employees_store          ON employees(store_id);
CREATE INDEX idx_products_category        ON products(category_id);
CREATE INDEX idx_products_supplier        ON products(supplier_id);
CREATE INDEX idx_orders_customer          ON orders(customer_id);
CREATE INDEX idx_orders_store             ON orders(store_id);
CREATE INDEX idx_orders_promotion         ON orders(promotion_id);
CREATE INDEX idx_orders_date              ON orders(order_date);
CREATE INDEX idx_items_order              ON order_items(order_id);
CREATE INDEX idx_items_product            ON order_items(product_id);
CREATE INDEX idx_payments_order           ON payments(order_id);
CREATE INDEX idx_shipments_order          ON shipments(order_id);
CREATE INDEX idx_returns_item             ON returns(order_item_id);
```